Introduction

Digit-manipulation problems show up constantly in coding interviews because they test something narrower and more useful than clever algorithms: can you carefully translate a plain-English rule into precise arithmetic without introducing bugs along the way? LeetCode 3622, Check Divisibility by Digit Sum and Product, is exactly this kind of problem. The algorithm itself is simple — you will not need a single advanced data structure — but the problem rewards engineers who read the rule carefully instead of skimming it and guessing.

That distinction matters in interviews. A candidate who jumps straight to code often builds something that only handles the "obvious" case, then stumbles when a zero digit or a single-digit input breaks their assumptions. This problem is a good filter for exactly that habit, which is why it's worth understanding deeply rather than just memorizing the solution.

Problem Overview

You're given a positive integer `n`. Break `n` into its individual digits. Compute two values from those digits:

- **Digit sum** — add every digit together.
- **Digit product** — multiply every digit together.

Add the digit sum and the digit product into one combined total. Then check whether `n` divides evenly into that total. If there's no remainder, return `true`. Otherwise, return `false`.

The key detail — and the one people misread — is that you don't check divisibility against the sum alone, or the product alone. You check it against **the sum of the two**, computed once, checked once.

Example

Take `n = 99`. Its digits are `9` and `9`.

- Digit sum: `9 + 9 = 18`
- Digit product: `9 × 9 = 81`
- Combined total: `18 + 81 = 99`
- Check: `99 % 99 == 0` → `true`

It's a satisfying example because the combined total lands exactly back on `n`, but don't let that coincidence suggest the two pieces are interchangeable. Neither `18` nor `81` divides `99` on its own — only the sum of the two does. That's the rule in action, not a shortcut around it.

A second example makes the "zero digit" case concrete: `n = 105`. Digits are `1`, `0`, `5`.

- Digit sum: `1 + 0 + 5 = 6`
- Digit product: `1 × 0 × 5 = 0`
- Combined total: `6 + 0 = 6`
- Check: `105 % 6 = 3` → `false`

The zero digit didn't break anything — it just zeroed out the product's contribution, leaving the sum to carry the check on its own.

Intuition

The first thing an experienced engineer does with a problem like this is separate what's *fixed* from what's *computed*. `n` is fixed — you're not allowed to change it. What you're computing is a single number (sum + product) built entirely from `n`'s own digits, and then you're asking a basic divisibility question about it.

That reframing immediately tells you the shape of the solution: this is a single pass over the digits of `n`, accumulating two running values, followed by one final check. There's no need to store the digits in a list, reverse a string, or recurse — you just need two accumulators and a loop.

The next thing worth noticing, before writing any code, is a boundary case that trips people up: what if a digit is `0`? Some engineers reflexively add a guard — "if the digit is zero, skip updating the product, because multiplying by zero would ruin everything." But look at what's actually being divided: it's the *sum* of digit sum and digit product, not the product by itself. Since `n` is a positive number, it has at least one digit, and the digit sum of a positive number's digits is always at least `1`. So even if the product collapses entirely to zero, the sum keeps the combined total above zero. No defensive check needed — the math already handles it.

This is a useful habit to build: before adding a guard clause, ask whether the guard is protecting against something that can actually happen given the other constraints of the problem. Here it can't, so the guard would just be unnecessary code.

Brute Force Solution

There isn't really a slower alternative here worth calling "brute force" in the traditional sense — this problem doesn't have a naive recursive or exponential approach to avoid. The one variant worth mentioning is converting `n` to a string and iterating over its characters instead of peeling digits arithmetically.

Idea: Convert `n` to a string, iterate over each character, convert each back to an integer, and accumulate sum and product.

Advantages: Slightly more readable to some engineers; avoids manual `% 10` / `// 10` arithmetic.

Disadvantages: String conversion and repeated `int()` casting add overhead that's completely unnecessary given how easy digit extraction is with arithmetic. It also encourages a habit — reaching for string conversion by default — that doesn't generalize well to problems with larger numeric constraints.

Time complexity: O(d) where `d` is the number of digits. **Space complexity:** O(d) for the string representation.

The arithmetic approach below has the same time complexity but true O(1) space, so it's the better default.

Optimal Solution

The optimal approach processes `n`'s digits one at a time using two well-worn tools:

- `n % 10` gives you the last digit.
- `n // 10` drops that last digit (integer division).

You keep a running `sum` (starting at `0`) and a running `product` (starting at `1`, since multiplying by zero at the start would erase everything). On each loop iteration:

1. Extract the last digit with `% 10`.
2. Add it to `sum`.
3. Multiply it into `product`.
4. Shrink the working number with `// 10`.

The loop ends when the working number reaches `0` — every digit has been visited exactly once. At that point, add `sum` and `product` together and check `n % (sum + product) == 0`.

Step	temp	digit	sum	product
start	99	—	0	1

Step	temp	digit sum	product
1	9	9	9
2	0	9	18	81

After the loop: `sum + product = 99`, and `99 % 99 == 0` → `true`.

Note that we work on a copy of `n` (call it `temp`) rather than mutating `n` itself, since the original value of `n` is needed for the final divisibility check.

Python Code

```
def check_divisibility(n: int) -> bool:
    """Return True if n is divisible by (digit sum + digit product) of n."""
    digit_sum = 0
    digit_product = 1
    temp = n

    while temp > 0:
        digit = temp % 10
        digit_sum += digit
        digit_product *= digit
        temp //= 10

    total = digit_sum + digit_product
    return n % total == 0
```

Code Walkthrough

- `digit_sum = 0`: additive identity — the starting point for a running sum.
- `digit_product = 1`: multiplicative identity — starting at `0` would zero out every digit multiplied into it.
- `temp = n`: a disposable copy, since `n` is needed intact for the final check.
- `while temp > 0`: loop continues as long as there are unprocessed digits.
- `digit = temp % 10`: extracts the last digit of the current `temp`.
- `digit_sum += digit` and `digit_product *= digit`: fold the digit into both running accumulators in the same pass.
- `temp //= 10`: drops the digit just processed, guaranteeing the loop terminates since `temp` shrinks by roughly a factor of 10 each iteration.
- `total = digit_sum + digit_product`: the one combination step, performed exactly once.
- `n % total == 0`: the one comparison, performed exactly once.

Dry Run

Let's trace `n = 84` from the video's opening hook.

temp	digit	digit_sum	digit_product
84	4	4	4
8	8	12	32
0	—	12	32

`total = 12 + 32 = 44`. Check: `84 % 44 = 40` → not zero → `false`.

So despite the video's teaser, `84` does **not** divide evenly by `44` — the point of that opening was to get you computing digit sum and digit product by hand, not to claim `84` passes the check.

Complexity Analysis

Time complexity: O(log n), or equivalently O(d) where `d` is the number of digits in `n`. Since `n ≤ 1,000,000`, that's at most 7 digits — the loop runs at most 7 times regardless of the exact value.

Space complexity: O(1). Only a handful of integer variables (`digit_sum`, `digit_product`, `temp`, `digit`) are used, independent of the size of `n`.

Both bounds are tight because every digit is visited exactly once, and no auxiliary data structure grows with input size.

Alternative Solutions

The only reasonable alternative is the string-based version described in the brute force section — converting `n` to `str(n)` and iterating over characters with `int(ch)`. It's functionally equivalent and has the same time complexity, but trades O(1) space for O(d) space and adds string/int conversion overhead. For a constraint this small (≤7 digits) the performance difference is negligible, but the arithmetic version is the more idiomatic choice for digit-manipulation problems in general, and that habit pays off on problems with much larger digit counts.

Edge Cases

- **Single-digit `n` (e.g., `n = 7`):** digit sum and digit product both equal `7`, so total is `14`. `7 % 14 = 7` → `false`. In fact, any single-digit number always produces a total of exactly double itself, so single-digit inputs can never return `true`.

- `n` **containing a zero digit (e.g.,** `n = 105`**):** the product term vanishes, but the sum still contributes, so the check still runs correctly with no special-casing.
- **Minimum input,** `n = 1`**:** digit sum `1`, digit product `1`, total `2`. `1 % 2 = 1` → `false`.
- **Maximum input,** `n = 1,000,000`**:** six zero digits collapse the product to `0`, digit sum is `1`, total is `1`. `1,000,000 % 1 = 0` → `true`.
- **All digits identical (e.g.,** `n = 22`**):** digit sum `4`, digit product `4`, total `8`. `22 % 8 = 6` → `false`.

Common Mistakes

1. **Adding a zero-guard for the digit product.** As covered above, this is unnecessary — the sum term guarantees the total is never zero, so there's nothing to divide by zero.
2. **Checking divisibility against the sum and product separately** instead of their combined total. The problem requires one combined check, not two independent ones.
3. **Initializing** `digit_product` **to** `0` **instead of** `1`**.** This silently zeroes out the entire product on the first multiplication.
4. **Mutating** `n` **directly instead of using a copy.** Since `n` is needed for the final modulo check, tearing it apart in the loop leaves nothing to check against.
5. **Off-by-one errors in the loop condition**, such as using `while temp >= 0`, which causes an infinite loop or an extra spurious "digit" of `0`.
6. **Forgetting to handle the loop when** `n` **itself is single-digit**, assuming the while loop needs at least two iterations to function correctly (it doesn't — one iteration handles it fine).

Interview Questions

- What would you need to change if `n` could be negative?
- How would your solution change if you needed the digit *count* as well as sum and product?
- Can this be solved recursively instead of iteratively? What are the tradeoffs?
- How would you adapt this if the base were not 10 (e.g., digit sum/product in base 16)?
- What's the largest `n` for which digit_product could overflow in a language without arbitrary-precision integers, and how would you defend against that?

Similar Problems

- **LeetCode 1281 — Subtract the Product and Sum of Digits of an Integer:** nearly identical digit extraction logic, just a different final combination step.

- **LeetCode 258 — Add Digits:** repeatedly sums digits until a single digit remains, reinforcing the peel-and-shrink pattern.
- **LeetCode 2520 — Count the Digits That Divide a Number:** same digit-peeling loop, different per-digit check.
- **LeetCode 9 — Palindrome Number:** uses the same `% 10` / `// 10` mechanics to process digits without converting to a string.
- **LeetCode 231 — Power of Two:** not digit-based, but a good companion problem for practicing "check a numeric property, then compare once" reasoning.

Key Takeaways

Digit-manipulation problems live and die on careful reading, not clever algorithms. The pattern here — peel with `% 10`, shrink with `// 10`, accumulate as you go, combine and compare exactly once — generalizes to a huge class of interview problems. The real skill being tested isn't the loop; it's resisting the urge to add unnecessary guards for cases the math already rules out, and resisting the urge to combine values in ways the problem statement didn't actually ask for.

Watch the Video

Prefer to see this traced by hand, digit by digit, with the two quick quizzes included? Watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series focused on making coding interview problems and core computer science concepts genuinely click — not just memorized, but understood well enough to explain to someone else. Every episode builds intuition first and code second, because that order is what actually sticks in an interview room.

Call To Action

If this walkthrough helped the logic click, the biggest favor you can do is like the video on YouTube and subscribe — it directly helps more developers find these breakdowns. For deeper dives delivered straight to your inbox, subscribe to the Substack newsletter. And if you spotted a cleaner approach or a case this didn't cover, drop it in the comments — sharing this with someone prepping for interviews helps them too.

Quick Review

Digit-sum/digit-product divisibility check — what's the trigger?

When a problem needs both sum and product of digits, extract digits with %10 and /10 in one loop, accumulate both.

Time complexity of checking divisibility by digit sum and product?

O(log10 n) — proportional to the number of digits in n.

Space complexity for this digit-peeling approach?

O(1) — just a few running accumulators, no extra storage.

Tricky edge case: what happens with a zero digit?

Product becomes 0 permanently; check n % 0 would crash — but n % (sum+product) is still safe since sum>0 covers it.

Why check n % (sum + product) instead of two separate checks?

Problem defines divisibility by the combined value sum+product, not by each independently — misreading this gives wrong condition.

Follow-up: how would you handle n=0 in this problem?

Constraints exclude n=0 (n≥1), so no digit-extraction loop guard is needed for that case.